

Jarvis AI OS

מסמך אפיון מלא - Vision, Architecture, Agents, Capabilities, MCPs, Security & Roadmap

מבוסס על סשן האפיון הנוכחי | 01.06.2026

חזון קצר

Jarvis הוא שכבת AI מקומית מעל המחשב: קליינט אחד, מנוע פקודות קשיח, שליטה קולית, סוכנים קבועים וזמניים, גישה מבוקרת לקבצים, Connectors/MCPs, זיכרון שקוף, Workflows, Sandbox, ניטור משאבים ויכולת הרחבה עתידית דרך Plugins.

Prepared for: Shay Marko

תוכן עניינים

1. תקציר מנהלים
2. גבולות המערכת והחזון
3. תרשים זרימה וארכיטקטורה מלאה
4. שכבות קלט ו-Input Router
5. Slash Commands ו-Command Engine
6. Jarvis Brain / Orchestrator
7. Agents קבועים וזמניים
8. Capabilities
9. MCPs / Connectors
10. RAG ו-Memory, Knowledge Base
11. Sandbox ו-Permissions, Approval Center, Secrets Vault
12. Durable Tasks ו-Workflows, Scheduler
13. Agent Debugger ו-System Monitor, Observability
14. File Experience ו-Client
15. SDKs ו-Java/Spring Boot מול Node
16. מבנה מודולים מומלץ
17. Roadmap לפי שלבים
18. סיכונים, שאלות פתוחות והמלצות יישום
19. נספחים: רשימות מלאות של פקודות, Agents, Capabilities ו-Connectors

1. תקציר מנהלים

המסמך הזה מגדיר את Jarvis כמערכת AI OS מקומית: לא עוד צ'אט בלבד, אלא שכבת הפעלה חכמה שמחברת בין המשתמש, המחשב המקומי, אפליקציות, קבצים, פקודות קוליות, סוכנים, מודלים, שירותים חיצוניים, Workflows, זיכרון והרשאות.

המטרה המרכזית היא לבנות מערכת שניתנת להרחבה לאורך זמן: היום היא פותחת קבצים, מריצה פקודות, מנהלת משימות וסוכנים; מחר היא יכולה לקבל Plug-in חדש, Connector חדש, MCP חדש, Agent חדש או Capability חדשה בלי לשבור את הליבה.

עיקרון עליון

Jarvis owns the architecture. SDKs are adapters כלומר: הארכיטקטורה, ההרשאות, ה-Agent runtime, הזיכרון, הפקודות והאורקסטרציה הם שלך; SDKs חיצוניים משמשים רק כמתאמים טכניים ל-API, MCP, OAuth, Browser Automation וכדומה.

מטרות המערכת

- לייצר קליינט אחד שבו ניתן לפתוח, לצפות, לערוך ולנתח קבצים מקומית.
- לאפשר שליטה באמצעות צ'אט, פקודות קוליות ו-Slash Commands קשיחות.
- לתזמר Agents קבועים וזמניים לפי סוג המשימה.
- לבצע פעולות על המחשב בצורה מבוקרת: קבצים, terminal, browser automation, workflows, אפליקציות.
- לחבר את Jarvis לשירותים חיצוניים דרך MCPs/Connectors כמו Gmail, GitHub, Slack, Notion, Google Drive, Calendar ועוד.
- לנהל זיכרון אישי שקוף לחלוטין למשתמש, כולל צפייה, עריכה, הוספה ומחיקה.
- להציג ניטור משאבי מחשב, agent observability-I tasks, agents, token/cost, connector health.
- להגן על המשתמש באמצעות Audit Log-I Permissions, Approval Center, Secrets Vault, Sandbox Runtime מלא.

2. גבולות המערכת והחזון

2.1 הגדרת המוצר

Jarvis הוא Local AI Operating Layer: שכבה מעל מערכת ההפעלה שמספקת ממשק עבודה אחד, AI Brain, Command Engine, Agent Runtime, גישה מבוקרת לקבצים, חיבור לשירותים חיצוניים ויכולת ביצוע פעולות אמיתיות על המחשב.

2.2 מה Jarvis אינו אמור להיות בשלב הראשון

- לא מערכת הפעלה מלאה שמחליפה macOS/Windows/Linux.
- לא Agent אוטונומי בלתי מוגבל שמורשה לבצע כל פעולה ללא אישור.
- לא תלוי ב-SDK אחד שמנהל את כל המערכת במקום הקוד שלך.
- לא מערכת עם גישה חופשית לכל הקבצים במחשב ללא Blocked Paths-I Allowed Folders.

2.3 עקרונות תכנון

עיקרון	משמעות מעשית
Local-first where possible	פעולות וקבצים מקומיים נשארים בשליטת המשתמש; מודלים ענניים משמשים לפי צורך והרשאה.
Deterministic commands first	Slash Commands מבצעים פעולה מוגדרת מראש ללא פרשנות AI חופשית.
Human-in-the-loop	פעולות רגישות עוברות Approval Center לפני ביצוע.
Extendable architecture	Connectors-I Agents, Capabilities, Commands, MCPs ניתנים להוספה בהמשך דרך Registry/Plugin SDK.
Transparent memory	המשתמש רואה, עורך, מוסיף ומוחק כל פריט זיכרון.

Observable agents	כל Agent run מתועד: למה הופעל, אילו כלים קרא, כמה זמן לקח, מה נכשל ומה עלה.
Sandbox before risk	קוד או פקודות מסוכנות רצים קודם ב-Sandbox מבודד, ורק לאחר אישור מקודמים לסביבה אמיתית.

3. תרשים זרימה וארכיטקטורה מלאה

התרשים הבא מציג את הזרימה המלאה: מהמשתמש, דרך Input Router, Command Engine או Jarvis Brain, משם ל-Outputs. Agents, Capabilities, Knowledge Base, MCPs/Connectors, Monitoring, Governance, ולבסוף.

Jarvis AI OS – תרשים זרימה מלא

המשתמש

קוליות / קודים
צ'אט טקסטואלי
Slash Commands / פקודות שישות

STT → קולי
פלט קולי → TTS = Spoken Summary

Input Layer

Voice Input | Chat Input | Slash Command Input

Input Router

- Workflow בדרך ישיר
- Approval בדרך
- Sandbox בדרך
- האם נדרש ביטוי עיצור
- האם זו בקשה מורכבת
- האם קיימת חבזת AI קטנה
- Model Router / Knowledge Base רלוונטי

A. Command Engine (מרחב הפקודה)

- Shell, Files, Memory, Resources, Tasks, Agents
- Settings, Status, Openapp, Terminal, Apps, Itop
- Workflows, Approve, AIs, Sandbox, Plugins, Models
- Secrets, Scheduler, Searchall, Debugger, Explorer, Assets
- Clipboard, API
- Command Registry, Command Handlers, Voice Aliases, Permission Guard
- Command Parser, Alias Mapping, Command Audit

B. Jarvis Brain / Orchestrator

- Intent Understanding, Planner, Task Manager
- Agent Registry, Progress Updates, Decision Engine
- Workflow Planner, Task Decomposition, Model Router
- Context Builder, Policy Engine, Tool / Capability Selector
- Jarvis coordinates everything, decides who handles what, keeps me updated, and makes sure nothing falls through.

Model Router / Model Manager

- Local model, Cloud model
- Privacy-aware routing, Cost-aware routing
- Latency-aware routing, Fallback model
- Multimodal model, Embedding model
- Cost, Latency, Quality

Workflow Builder / Automation Designer

- Triggers, Conditions, Steps, Agent Assignment
- Human Approval, Retries, Logs, Repeating Schedules
- Trigger reason: time, voice, email received, file changed, webhook

Approval Center / Human-in-the-Loop

- Pending Approvals, Approve / Deny, Risk Level
- Why Jarvis asks: Prioritize Before Action, Remember the decision
- Risk Commands, Terminate, File Delete, Send Email, Git Push, Deployment Slack Send

Plugin / Extension SDK / Marketplace

- Custom Commands, Custom Agents, Custom Connectors, Custom Capabilities
- UI Panels, Settings, Permissions, MCP Servers

Permanant Agents

- Product / Spec Agent, Backend Agent, Frontend Agent, Test Agent, Code Review Agent
- UI Screenshot QA Agent, DevOps / Cloud Agent, Marketing Agent, Social Media Agent, Email Agent, Calendar & Schedule Agent
- Trading News Agent, Digital Marketing Agent, Research Agent, Customer Support Agent, Code & Bug Fix Agent
- Security / Permission Agent, Browser Automation Agent, Knowledge Librarian Agent, Data Analyst Agent
- Workflow Automation Agent, Meeting Assistant Agent, Finance / Budget Agent, Home / IoT Agent, Model Router Agent
- Evaluator / Critic Agent, Backup & Sync Agent, Connector Health Agent, Privacy Agent

Temporary Agents

- נתיבים דינמיים
- תתי-משימות
- Subtasks
- Parallel work
- Results merge back to Jarvis
- Ephemeral specialists

Secrets Vault / Credentials Manager

- Encrypted Tokens, GitHub, Refresh Tokens
- Power express secrets, code, Revoke Access
- Gmail, Slack, GitHub MCP, File browser, Outlook

Sandbox Runtime / Isolated Execution

- Run generated code safely, Isolated Ecosystem, Temporary environments
- Package install inside sandbox, Output capture, Promote after approval

Agent Debugger / Observability

- Agent Run Trace, Prompt / Test Trace, Replay, Compare Runs
- Failures, Timing, Tokens, Cost
- Reassuring Summary

Knowledge Base / RAG

- Document Indexing, Semantic Search, Vector DB, Project Knowledge, Codebase Knowledge, Personal Notes, Source Citations, RAG Retrieval

Capabilities

- File System Capability: Jarvis Explorer, images, video, audio, text, Markdown, PDF, code, CSV-Excel, JSON-YAML, emails, notes, logs
- Browser / Google Search, Summarization, Notification Capability, Media Generation, Screenshot Comparison, System Resource Monitoring, Local Actions / Automation

Execution & Safety

- Sandbox Execution, Approval Center, Durable Task Engine, Secrets Vault

Analysis & Understanding

- OCR / Screen Understanding, Vector Search, Knowledge Base / RAG, Summarization

Automation & Extensibility

- Workflow Builder, Scheduler / Cron Engine, Webhook Server, API Server for Jarvis

MCPs / Connectors

- Productivity: Gmail, Slack, Google Drive, Notion, Google Search, Apple Notes, Reminders, Contacts
- Developer: GitHub, JetBrains IDE, Jira / Linear / Trello, Discord / Telegram, PaaS
- Data / Infrastructure: Filesystem MCP, SQLite MCP, Docker MCP, Kubernetes / OpenShift MCP
- Communication & Media: Browser / Playwright MCP, RSS / News APIs, Spotify / YouTube
- Home / IoT: Home Assistant, Maps / Location
- Finance / Business: Stripe / PayPal, Webhook Connector

System Monitor / Task Manager

- CPU, RAM, Disk, Network, Processes, Jarvis Health, Active Agents, Running Tasks
- Cost / Token Monitor, Scheduler Jobs, Connector Health, Task Queue, Durable Tasks, Local Model Load GPU

Observability / Agent Debugger

- Agent Run Trace, Prompt / Test Trace, Replay, Compare Runs
- Failures, Timing, Tokens, Cost
- Reassuring Summary

Outputs

- חבזת, תשובה, תוצאות מסומנות, סקריפט, פעילות, מערכת, עדכון סטטוס / הודעות, סיכום מצב, Workflow

מקראי וחיבורים

- חיבור ראשוני
- ← חיבור שניוני

3.1 זרימה כללית

- המשתמש מפעיל את Jarvis באמצעות קול, צ'אט או Slash Command.
- Input Router מזהה האם מדובר בפקודה קשיחה, בקשת AI, workflow, פעולה שדורשת אישור, פעולה שדורשת sandbox או פעולה שדורשת model routing.
- פקודות קשיחות עוברות ל-Command Engine; בקשות מורכבות עוברות ל-Jarvis Brain / Orchestrator.
- ה-Jarvis Brain מפרק את המשימה, בוחר Model-י Agents, Capabilities, Connectors מתאים.
- פעולות רגישות עוברות ל-Approval Center-י Permission Guard.
- פעולות קוד/terminal מסוכנות יכולות לרוץ ב-Sandbox Runtime לפני השפעה על מערכת אמיתית.
- כל תהליך נרשם ב-Task History, Audit Log, ו-Agent Observability.
- הפלט חוזר בצ'אט, בקול, בחלונות פנימיים, בקבצים שנוצרו, בהתראות או בדוחות workflow.

4. שכבות קלט ו-Input Router

4.1 סוגי קלט

סוג קלט	דוגמה	יעד ראשוני
Voice Input	"Jarvis, show my files"	STT -> Input Router
Chat Input	"תסכם לי את המיילים החשובים"	Input Router -> AI Brain
Slash Command Input	jfiles/	Input Router -> Command Engine
Trigger חיצוני	email received / webhook / file changed	Workflow Builder / Task Engine

4.2 החלטות Input Router

- האם זו פקודה קשיחה?
- האם זו בקשת AI?
- האם נדרש Agent?
- האם נדרשת Capability?
- האם נדרש Workflow?
- האם נדרש Approval?
- האם נדרש Sandbox?
- האם נדרש Model Router?
- האם יש Knowledge Base רלוונטי?
- האם צריך Connector חיצוני או פעולה מקומית?

5. Command Engine ו-Slash Commands

Command Engine מטפל בפקודות קשיחות שאינן דורשות reasoning חופשי. כל פקודה מפעילה פעולה אחת מוגדרת מראש דרך handler ייעודי. Slash command always wins: אם הקלט מתחיל בפקודת slash מוכרת, הוא לא עובר לפרשנות AI כללית.

5.1 מבנה Command Definition

שדה	תיאור
name	שם פנימי של הפקודה.
slash	הצורה שבה המשתמש כותב את הפקודה, למשל /help.
voiceAliases	ביטויים קוליים שמפעילים את אותה פקודה.
description	הסבר למשתמש בחלון /help.
parameters	פרמטרים שהפקודה מקבלת.
requiredPermissions	הרשאות נדרשות לפני ביצוע.

handler	מחלקת הקוד שמבצעת את הפעולה.
visibleInHelp	האם להציג ב-.help/
category	system, files, agents, memory, workflows, dev, monitoring וכו'.

5.2 רשימת פקודות ראשונית

פקודה	פעולה
help/	מציג חלון עזרה עם הפקודות הזמינות.
jfiles/	מציג את הקבצים ב-Jarvis Explorer.
memory/	פותר Memory Manager.
resources/	פותר System Resource Dashboard.
tasks/	מציג משימות פתוחות.
agents/	מציג Agents פעילים וקבועים.
settings/	פותר הגדרות Jarvis.
status/	מציג מצב מערכת כללי.
<openapp <name/	פותר אפליקציה מקומית לפי הרשאות.
<terminal <command/	מריץ פקודת terminal לפי Approval-I Permission Guard.
logs/	מציג Audit Logs.
stop/	עוצר פעולה שרצה כרגע.
workflows/	מציג/מנהל Workflows.
approve/	פותר Approval Center.
kb/	פותר Knowledge Base Manager.
sandbox/	מציג או מפעיל Sandbox Runtime.
plugins/	מציג Plugin / Extension SDK / Marketplace.
models/	מציג Model Manager / Model Router.
secrets/	פותר Secrets Vault.
scheduler/	מציג Scheduler / Cron jobs.
searchall/	חיפוש גלובלי מאושר במחשב וב-KB.
debugger/	פותר Agent Debugger.
replay/	מריץ replay של Agent run או workflow.
costs/	מציג Cost / Token Monitor.
clipboard/	פעולות clipboard מורשות.
api/	מציג API Server / Webhook info.

Jarvis Brain / Orchestrator .6

Jarvis Brain הוא הרכיב שמבין כוונה, מתכנן, מחלק משימות, בוחר סוכנים, מפעיל capabilities, מנתב מודלים ומוודא שהמשימה מסתיימת. הוא לא מריץ הכול בעצמו אלא מתזמר את המערכת.

תת-רכיב	אחריות
Intent Understanding	הבנת בקשת המשתמש וסיווג סוג המשימה.
Planner	בניית תוכנית פעולה.
Task Manager	ניהול מצב המשימה, תתי-משימות והתקדמות.
Agent Registry	איתור Agents זמינים ומתאימים.
Progress Updates	עדכונים למשתמש בצ'אט/קול/התראות.
Decision Engine	בחירת פעולה נכונה לפי מצב, הרשאות וסיכון.
Workflow Planner	בניית workflow כאשר המשימה דורשת רצף צעדים.
Task Decomposition	פירוק משימה לתתי-משימות.
Model Router	בחירת מודל לפי פרטיות, עלות, latency, איכות וסוג משימה.
Context Builder	איסוף context מזיכרון, קבצים, KB, שיחה ו-connectors.

Policy Engine	החלת policies של הרשאות, פרטיות וסיכון.
Tool / Capability Selector	בחירת connector או capability לביצוע בפועל.

7. Agents קבועים וזמניים

Permanent Agents 7.1

Permanent Agents הם תחומי אחריות קבועים במערכת. הם לא בהכרח processes נפרדים, אלא יחידות מומחיות עם context-ו prompt, tools, permissions משלהן.

Agent	אחריות
Product / Spec Agent	יצירת מסמכי אפיון, user stories, acceptance criteria ודרישות מוצר.
Backend Agent	כתיבת שירותי backend, API, persistence, business logic.
Frontend Agent	פיתוח client, UI, components, state management.
Test Agent	כתיבת unit/integration/E2E tests והרצתם.
Code Review Agent	בדיקת איכות קוד, naming, architecture, security smells.
UI Screenshot QA Agent	השוואת screenshots מול דרישות ועיצוב.
DevOps / Cloud Agent	.Docker, CI/CD, deployment, cloud, OpenShift/Kubernetes
Debug Agent	איתור ותיקון באגים, קריאת logs, reproduction.
Email Agent	קריאת מיילים, סיכום מיילים חשובים, הצעות תגובה.
Calendar & Schedule Agent	יומן, פגישות, reminders, תיאום משימות.
Trading News Agent	בדיקת חדשות וכתבות למסחר פונדמנטלי.
Digital Marketing Agent	שיווק דיגיטלי, קמפיינים, copy, funnels.
Social Media Agent	פוסטים, תזמון, ניתוח engagement.
Customer Support Agent	מענה ללקוחות, FAQ, tickets.
Code & Bug Fix Agent	התמקדות בפתירת באגים ופיתוח קוד.
Security / Permission Agent	בדיקת סיכונים, הרשאות, secrets-I command risk.
Browser Automation Agent	ניווט באתרים, מילוי טפסים, scraping מבוקר, בדיקות UI.
Research Agent	מחקר, השוואת מקורות, סיכומים וציטוטים.
Knowledge Librarian Agent	ארגון מסמכים, notes, KB, tags ומקורות ידע.
Data Analyst Agent	ניתוח CSV/Excel/DB, גרפים, insights.
Workflow Automation Agent	בניית workflows אוטומטיים ו-recurring automations.
Meeting Assistant Agent	תמלול, סיכום פגישות, action items.
Finance / Budget Agent	ניהול הוצאות, חשבוניות, תקציב אישי/עסקי.
Home / IoT Agent	שליטה בבית חכם דרך Home Assistant בעתיד.
Model Router Agent	בחירת מודלים והערכת עלות/איכות/latency.
Evaluator / Critic Agent	בדיקת איכות תוצרים לפני החזרה למשתמש.
Backup & Sync Agent	גיבוי, שחזור וסנכרון קבצים.
Connector Health Agent	בדיקת תקינות Gmail/GitHub/Notion וכו'.
Privacy Agent	זיהוי מידע רגיש לפני שליחה למודל חיצוני.

Temporary Agents 7.2

Temporary Agents נוצרים דינמית לפי משימה. הם יכולים לפעול במקביל, להתמקד בתת-משימה, ואז להחזיר תוצאה ל-Jarvis Brain שממזג את התוצרים.

- נוצרים דינמית לפי סוג המשימה.
- משמשים כ-ephemeral specialists.
- מחולקים לפי subtasks.
- יכולים לעבוד במקביל.

- מחזירים תוצאות ל-Jarvis Brain לצורך merge, validation output-I סופי.

Capabilities .8

Capabilities הן הפעולות שהמערכת יודעת לבצע בפועל. Agents חושבים ומתכננים; Capabilities מבצעות קריאה, כתיבה, חיפוש, הרצה, פתיחת אפליקציה, ניטור וכדומה.

Capability	תיאור
File System Capability	קריאה/כתיבה/יצירה/מחיקה מבוקרת לפי Allowed Paths-I Blocked Paths-I Folders
Jarvis Explorer	כונן/תיקיית הבית של Jarvis עם Downloads, Documents, Projects, Media, Notes, Generated וכו'.
Internal File Viewers / Editors	פתיחת תמונות, וידאו, אודיו, טקסט, Markdown, PDF, קוד, CSV/Excel, JSON/YAML, מיילים, פתקים ולוגים בתוך הקליינט.
Terminal Capability	הרצת פקודות shell תחת הרשאות, sandbox-I approval לפי רמת סיכון.
App Launcher	פתיחת אפליקציות מקומיות לפי בקשת המשתמש.
Browser / Google Search	חיפוש מידע וסיכום תשובות.
Browser Automation	ניווט באתרי web, מילוי טפסים, scraping ובדיקות UI.
Summarization	סיכום מיילים, מסמכים, קבצים, חדשות ופגישות.
Notification Capability	התראות, עדכוני סטטוס notification center-I.
Media Generation	יצירת תמונות/וידאו/אודיו/נכסי social לפי prompt קולי או טקסטואלי.
Screenshot Comparison	השוואת UI screenshots מול דרישות ועיצוב.
System Resource Monitoring	CPU/RAM/Disk/Network/Processes/Agent usage.
Local Actions / Automation	פעולות מקומיות במחשב בצורה מורשית.
Sandbox Execution	הרצת קוד/פקודות בסביבה מבודדת לפני ביצוע אמיתי.
Workflow Builder	בניית תהליכים קבועים עם triggers, steps, conditions approvals-I.
Approval Center	ניהול אישורים לפני פעולות רגישות.
Durable Task Engine	שמירת מצב משימות, pause/resume/retry/recover.
Knowledge Base / RAG	אינדוקס מסמכים, חיפוש סמנטי ותשובות מבוססות מקורות.
Vector Search	חיפוש סמנטי על מסמכים, notes, קוד וידע אישי.
Model Router	בחירת מודל לפי סוג משימה, פרטיות, עלות, latency ואיכות.
Secrets Vault	ניהול per-connector -I tokens, OAuth, refresh, revoke permissions.
Plugin / Extension SDK	הוספת MCP -I Commands, Agents, Connectors, Capabilities servers.
Agent Debugger / Replay	.trace, replay, compare runs, failures, timing, tokens, cost
Cost / Token Monitor	מעקב עלויות, quality-I tokens, latency.
OCR / Screen Understanding	הבנת מסך, OCR, extracting UI text.
Clipboard Capability	קריאה/כתיבה ל-clipboard לפי הרשאות.
Global Search Across Computer	חיפוש גלובלי רק בתיקיות מאושרות.
Backup / Restore	גיבוי ושחזור תצורות, זיכרון, קבצים ו-workflows.
Webhook Server	קבלת events ממערכות חיצוניות.
Scheduler / Cron Engine	הרצת משימות מתוזמנות.
Local Database Capability	גישה ל-SQLite/Postgres/MongoDB לפי הרשאה.
API Server for Jarvis	API פנימי/חיצוני להפעלת Jarvis מאפליקציות אחרות.

קטגוריה	Connectors / MCPs
Productivity	Gmail, Slack, Google Drive, Calendar, Notion, Google Search, WhatsApp Web, Outlook / Microsoft 365, Google Sheets, .Contacts Connector, Apple Notes / Reminders / Contacts
Developer	GitHub, Git MCP, GitHub MCP, Jira / Linear / Trello, Discord / .Telegram, Figma Connector
Data / Infrastructure	Filesystem MCP, Postgres MCP, SQLite MCP, MongoDB .MCP, Docker MCP, Kubernetes / OpenShift MCP
Communication & Media	Browser / Playwright MCP, RSS / News APIs, Spotify / .YouTube
Home / IoT	.Home Assistant, Maps / Location
Finance / Business	.Stripe / PayPal, Webhook Connector

RAG-1 Memory, Knowledge Base .10

פעולה	תיאור
View	המשתמש יכול לראות כל פריט זיכרון.
Add	המשתמש יכול להוסיף ידנית זיכרון.
Edit	המשתמש יכול לערוך זיכרון קיים.
Delete	המשתמש יכול למחוק זיכרון.
Source / Provenance	לכל זיכרון יש מקור, תאריך יצירה, עדכון ורמת confidence.
Sensitivity	אפשר לסמן זיכרון כרגיש ולדרוש אישור לפני שימוש.
Expiration	אפשר להגדיר תוקף לזיכרון זמני.
Export / Backup	אפשר לייצא ולגבות את הזיכרון.

תפקיד	רכיב
מידע על המשתמש והעדפותיו.	Memory
מידע מתור קבצים/מסמכים/פרויקטים.	Knowledge Base

Task History	מה Jarvis עשה בעבר.
Audit Log	תיעוד פעולות, אישורים והרשאות.

11. Permissions, Approval Center, Secrets Vault ו-Sandbox

11.1 Permission Modes

Mode	התנהגות
Safe Mode	Jarvis רק קורא, מנתח ומציע. לא מבצע פעולות רגישות.
Assisted Mode	Jarvis מבצע פעולות לאחר אישור משתמש.
Autonomous Mode	Jarvis מבצע לבד רק בתחומים שאושרו מראש עם גבולות ברורים.

11.2 Approval Center

Approval Center הוא מסך מרכזי שמציג פעולות ממתינות לאישור, הסבר למה Jarvis רוצה לבצע אותן, רמת סיכון, preview לפני ביצוע, ואפשרות לזכור החלטה לפעולות דומות.

- Pending Approvals
- Approve / Deny
- Risk Level
- Why Jarvis asks
- Preview Before Action
- Remember this decision

פעולות שמומלץ לדרוש עבורן אישור: שליחת מייל, שליחת WhatsApp/Slack, מחיקת קובץ, שינוי קוד, git push, deployment, התקנת package, פקודת terminal מסוכנת, גישה לתיקייה חדשה, שינוי credentials.

11.3 Secrets Vault / Credentials Manager

- Encrypted Tokens
- OAuth
- Refresh Tokens
- Revoke Access
- Per-connector permissions
- Never expose secrets to LLM
- Audit secret access

11.4 Sandbox Runtime / Isolated Execution

Sandbox Runtime מאפשר ל-Jarvis להריץ קוד שנוצר על ידי AI או פקודות מסוכנות בסביבה מבודדת. התוצאה נאספת, נבדקת, ורק לאחר אישור ניתן לקדם שינוי לסביבה האמיתית.

- Run generated code safely
- Isolated filesystem
- Temporary environments
- Package install inside sandbox
- Output capture
- Promote after approval

12. Durable Tasks-ו Workflows, Scheduler

Workflow Builder מאפשר להגדיר תהליכים קבועים, כמו סיכום בוקר: מיילים חשובים, יומן, חדשות מסחר, משימות פתוחות וסיכום קולי קצר.

דוגמה	רכיב Workflow
-------	---------------

Triggers	.time, voice, email received, file changed, webhook
Conditions	אם הגיע מייל מכתובת X, אם CPU נמוך, אם יש פגישה היום.
Steps	קריאה, סיכום, בדיקת הרשאות, יצירת קובץ, שליחת התראה.
Agent Assignment	בחירת Email Agent, Calendar Agent, Research Agent וכו'.
Human Approval	עצירת flow לפני פעולה רגישה.
Retries	נסיונות חוזרים לפי policy.
Logs	תיעוד כל שלב.
Recurring Schedules	הרצה יומית/שבועית/לפי cron.

Durable Task Engine 12.1

- Save task state
- Resume after crash
- Pause / continue
- Retry failed step
- Show progress timeline
- Store intermediate results
- Recover unfinished workflows

13. Agent Debugger- System Monitor, Observability

13.1 System Monitor / Task Manager

Jarvis מציג נתוני Task Manager בזמן אמת בתוך הקליינט ומשתמש בהם גם לצורך החלטות: דחיית משימות כבדות, התראה על RAM/Disk, השהיית Agent בעייתי וכו'.

- CPU
- RAM
- Disk
- Network
- Processes
- Jarvis Health
- Active Agents
- Running Tasks
- Cost / Token Monitor
- Scheduler Jobs
- Connector Health
- Task Queue
- Durable Tasks
- Local Model Load
- GPU

13.2 Agent Debugger / Observability

- Agent Run Trace
- Prompt / Tool Trace
- Replay
- Compare Runs
- Failures
- Timing
- Tokens
- Cost

Reasoning Summary •

מטרת ה-Observability היא להפוך את Jarvis לשקוף: להבין למה Agent הופעל, איזה model נבחר, איזה tools נקראו, איפה נכשלו, וכיצד לשחזר או להשוות ריצות.

14. Client-File Experience

הקליינט הוא המקום שבו המשתמש עובד בפועל: צ'אט, חלונות פנימיים, Jarvis Explorer, מסכי memory, media-1 workflows, approvals, resources, agents, files, outputs.

Jarvis Explorer 14.1

Jarvis Explorer הוא root וירטואלי של Jarvis, לדוגמה: /Media, /Projects, /Documents, /Downloads, /Notes, /Emails, /Generated, /Screenshots, /Logs.

14.2 Allowed External Folders

בנוסף לתיקיית Jarvis Explorer, Jarvis יכול לגשת רק לתיקיות שהוגדרו בקובץ properties/config. למשל Downloads, Desktop, Documents. לכל תיקייה ניתן להגדיר read-only או read-write.

14.3 Internal Viewers / Editors

סוג קובץ	יכולת
תמונות	צפייה, הגדלה, סיבוב, metadata.
וידאו	נגן פנימי.
אודיו / מוזיקה	נגן פנימי-1-playlists.
טקסט / Markdown	צפייה, עריכה ושמירה.
קוד	viewer/editor לפי שפת קוד.
PDF	צפייה, חיפוש, סיכום AI.
CSV / Excel	טבלאות, ניתוח וגרפים.
JSON / YAML	viewer, formatting, validation.
מיילים	צפייה, סיכום, תגובה.
פתקים	ניהול notes פנימי.
לוגים	חיפוש, filtering וניתוח שגיאות.

15. החלטות טכנולוגיות: Java/Spring Boot מול Node ו-SDKs

15.1 Backend ראשי

ההמלצה: לבנות את ה-core/backend הראשי ב-Java + Spring Boot. זה ריאלי מאוד ומתאים במיוחד למערכת stateful, גדולה, ארוכת חיים, עם הרשאות, connectors, task orchestration, agents, audit logs, memory, monitoring-1.

אפשרות	יתרונות	חסרונות / הערות
Java + Spring Boot	יציבות, DI, טיפוסים, services, scheduling, security, persistence, enterprise architecture.	פחות SDKs מוכנים סביב חלק מעולמות AI לעומת TypeScript/Python; ייתכן צורך ב-sidecars.
Node / TypeScript	מהיר ל-MVP, ecosystem-1-Playwright, Electron, MCP TS, Claude ל-Agent SDK.	פחות מתאים כליבה יחידה למערכת governance/stateful כבדה אם רוצים ארכיטקטורה enterprise.
Hybrid	Java core + Node/Python sidecars לפי צורך.	דורש ניהול IPC/API בין רכיבים, אבל נותן את הטוב משני העולמות.

15.2 המלצה ארכיטקטונית

- Jarvis Core Backend: Java 21/25 + Spring Boot
- Electron או Jarvis Client: React + Tauri
- Claude Agent SDK / שימוש נקודתי ב-Claude Agent SDK אם יידרש.
- Python Sidecar: OCR, ML, local models, media tools במידת הצורך.
- Agent Runtime: לבנות לבד כחלק מהליבה של Jarvis.
- SDKs: להשתמש ב-adapters בלבד עבור API/MCP/OAuth/Streaming במקום לתת להם את הארכיטקטורה.

15.3 גישה ל-Agent SDK / Claude SDK

אפשר לא להשתמש ב-Claude Agent SDK ולבנות את ה-agent runtime לבד. זה אפשרי, אבל דורש יותר עבודה: agent loop, tool calling loop, context management, permissions, streaming, error handling, MCP integration, long-running tasks ו-session state

מה לבנות לבד	מה אפשר לקחת מ-SDK
Brain, Agent Runtime, Command Engine, Tool Registry, Permission Guard, Approval Center, Memory Manager, Task Manager, Workflow Engine, Sandbox Policy, UI Protocol	Claude API client, streaming helpers, MCP Java SDK/Spring AI MCP, Google/GitHub SDKs, Playwright SDK, OAuth libraries

16. מבנה מודולים מומלץ

מודול	תכולה
jarvis-core	brain, planner, orchestrator, agents, commands, capabilities, permissions, memory, tasks, workflows, audit
jarvis-ai	model-router, claude-client, openai-client, local-model-client, prompt-builder, tool-calling-loop, context-builder
jarvis-mcp	mcp-client, mcp-server, mcp-tool-adapter, mcp-resource-adapter
jarvis-connectors	gmail, github, notion, slack, google-drive, calendar, whatsapp, web ו-UI
jarvis-local	filesystem, terminal, app-launcher, system-monitor, sandbox, browser-automation
jarvis-api	.REST, WebSocket, SSE, webhook/API server
jarvis-client	.desktop/web client: React + Tauri/Electron

17. Roadmap לפי שלבים

שלב	תכולה
Phase 1 - Client + Command Engine	Jarvis Client, Jarvis Explorer, /help, /files, /settings, .Command Registry, Input Router
Phase 2 - File Access + Permissions	Allowed Folders, Blocked Paths, File viewer/editor, Audit Log, .Permission Guard
Phase 3 - Memory Manager	Memory screen, add/edit/delete/search/export, .source/provenance, memory audit
Phase 4 - System Monitor	CPU/RAM/Disk/Network, processes, Jarvis health, .agents/tasks runtime status, /resources
Phase 5 - Sandbox + Approval	Sandbox Runtime, Approval Center, risky actions policy, .terminal risk classification
Phase 6 - Agent System	Permanent agents, temporary agents, planner, task manager, .status updates
Phase 7 - Connectors & MCPs	Gmail, Calendar, GitHub, Notion, Drive, Slack, Search, MCP .framework
Phase 8 - Workflow Builder	Triggers, steps, conditions, schedules, durable tasks, .recurring automations

Phase 9 - Knowledge Base / RAG	Document indexing, vector DB, project/codebase knowledge, .citations
Phase 10 - Model Router + Observability	Model manager, cost/latency/quality, agent debugger, .replay/compare runs
Phase 11 - Media Generation + Advanced Automation	AI media generation, browser automation, social/customer .workflows, plugin marketplace

18. סיכונים, שאלות פתוחות והמלצות יישום

18.1 סיכונים מרכזיים

סיכון	מענה מומלץ
גישה רחבה מדי לקבצים	Allowed Folders, Blocked Paths, read-only/read-write .permissions, Audit Log
פקודות terminal מסוכנות	.Risk Classification, Approval Center, Sandbox first
זליגת secrets ל-LLM	Secrets Vault, masking, never expose secrets to model .context
Agent עושה פעולה לא נכונה	.Human-in-the-loop, replay/debugger, evaluator/critic agent
עלות מודלים גבוהה	.Model Router, Cost/Token Monitor, local model routing
מערכת גדולה מדי ל-MVP	פיתוח בשלבים, modular monolith, plugin interfaces רק אחר core יציב.
תלות ב-SDK חיצוני	SDKs as adapters; core architecture נשארת שלך.
UI עמוס מדי	Dashboard modular, panels נפתחים לפי /help context, navigation-I טובים.

18.2 שאלות פתוחות

- האם הקליינט יהיה Tauri או Electron?
- איזה DB ישמש לזיכרון, KB: PostgreSQL/SQLite/MongoDB-I tasks, audit
- איזה Vector DB מתאים לשלב ראשון?
- האם להריץ מודל מקומי על Mac mini M1 בשלב MVP או רק בהמשך?
- איך להגדיר risk levels לפקודות terminal ופעולות קובץ?
- איך ייראה Plugin Manifest פורמלי?
- איך לנהל permissions לפי user / agent / connector / command?
- מה רשימת הפקודות הראשונית שתוצא להכניס ל-/help כבר ב-MVP?

18.3 המלצת MVP

ה-MVP המומלץ: Java/Spring Boot core, React client, Jarvis Explorer, Command Engine, Allowed Folders, Permission Guard, Memory Manager, System Monitor, Audit Log, Approval Center, Sandbox placeholder, Agent Registry, Claude/OpenAI adapter-ו ראשוני. לאחר שהשלב עובד, להוסיף .Knowledge Base-ו Workflows, MCPs, Browser Automation

נספח א' - מודל הרשאות לפעולות רגישות

פעולה	ברירת מחדל מומלצת
קריאת קובץ ב-Jarvis Explorer	מותרת.
קריאת קובץ בתיקייה חיצונית מאושרת	מותרת לפי permission.
כתיבת קובץ	policy מוגדרת. IA Assisted Mode
מחיקת קובץ	דורש Approval.
שינוי קוד	דורש Approval או sandbox/test קודם.
שליחת מייל/WhatsApp	דורש Approval-I-preview.
git push	דורש Approval.
deployment	דורש Approval ורמת סיכון גבוהה.
התקנת package	דורש Approval; עדיף Sandbox קודם.
קריאת secrets	רק דרך Secrets Vault; לעולם לא לחשוף למודל.
פתיחת אפליקציה	לרוב מותרת, לפי allowlist.
הרצת terminal	לפי risk classification + Approval/Sandbox.

נספח ב' - דוגמת תצורת Allowed Folders

דוגמת YAML ל-config עתידי:

```
jarvis:
  filesystem:
    jarvisRoot: /Users/shay/JarvisExplorer
    allowedFolders:
      - name: system-downloads
        path: /Users/shay/Downloads
        permissions: read-write
      - name: desktop
        path: /Users/shay/Desktop
        permissions: read-only
      - name: documents
        path: /Users/shay/Documents
        permissions: read-write
    blockedPaths:
      - /System
      - /Library
      - /Applications
      - ~/.ssh
      - ~/.aws
      - ~/.config/secrets
```

נספח ג' - דוגמת מבנה זיכרון

```
{
  "id": "mem_001",
  "category": "preference",
  "title": "Comparison style",
  "content": "User prefers comparison tables when comparing items.",
  "source": "chat",
  "createdAt": "2026-06-01T00:00:00+03:00",
  "updatedAt": "2026-06-01T00:00:00+03:00",
  "confidence": 0.92,
  "visibility": "user-visible",
  "sensitivity": "normal",
  "expiresAt": null,
  "enabled": true
}
```

נספח ד' - הגדרת חזון רשמית

Vision Statement

Jarvis is a local AI operating system layer with a unified client, deterministic command engine, voice control, agent orchestration, controlled local file access, external connectors, personal memory management, system resource monitoring, workflow automation, sandboxed execution, model routing, knowledge base retrieval, plugin extensibility, and AI-powered automation. Jarvis acts as the central brain, creates or assigns agents dynamically, executes actions through controlled capabilities, keeps the user updated, and ensures that tasks are completed safely without anything falling through.

חזון בעברית

Jarvis הוא שכבת AI מקומית מעל המחשב, הכוללת קליינט אחד, מנוע פקודות קשיח, שליטה קולית, ניהול סוכנים, גישה מבוקרת לקבצים, חיבורים לשירותים חיצוניים, ניהול זיכרון אישי, ניהול משאבי מחשב, Workflows, Sandbox, ניתוב מודלים, Knowledge Base ויכולת הרחבה דרך Jarvis Plugins. משמש כמוח מרכזי שמבין את הבקשה, מחליט מי צריך לטפל בה, יוצר סוכנים זמניים בעת הצורך, מפעיל יכולות מערכת בצורה מבוקרת, מעדכן את המשתמש ודואג ששום דבר לא ייפול בין הכיסאות.